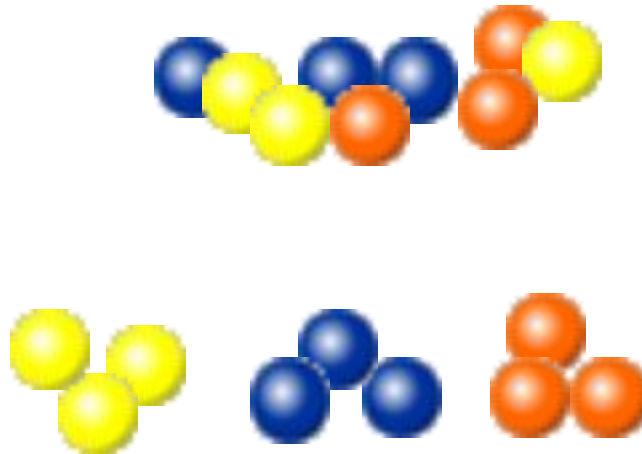


聚类



What is Cluster Analysis?

- Cluster: a collection of data objects
 - **Similar** to one another within the same cluster
 - **Dissimilar** to the objects in other clusters
- Cluster analysis
 - 根据数据的特征找出数据间的相似性，将相似的数据分成一个类。
- **Unsupervised learning**: no predefined classes

Clustering: Rich Applications and Multidisciplinary Efforts

- Typical applications
 - 作为一个独立的工具对数据分布进行分析
 - 可以作为其他算法（如分类等）的预处理步骤
 - Pattern Recognition
 - Spatial Data Analysis
 - Image Processing
 - Economic Science (especially market research)
 - WWW

Examples of Clustering Applications

- Marketing: 帮助市场分析人员从客户基本库中发现不同的客户群，并且用购买模式来刻画不同的客户群的特征
- Land use: 在地球观测数据库中用以确定相似地区
- City-planning: 根据房子的类型，价值，和地理位置对一个城市中的房屋分组

Quality: What Is Good Clustering?

- 好的聚类方法能产生高质量的聚类。所谓高质量，指：
 - 类中的对象高度相似
 - 类间的对象高度不相似
- 聚类的质量与什么有关？
 - 相似性度量及其实现方法
- 聚类方法的好坏也可以按照它是否能够发现更多的隐含模式来度量

Measure the Quality of Clustering

- **Dissimilarity/Similarity metric**: 相似性通常用距离函数来表示, typically metric: $d(i, j)$
- 距离函数的定义对于不同的数据通常是不同的, 如:
 - interval-scaled
 - Boolean
 - Categorical
 - ordinal ratio
 - vector variables.
- 根据应用背景或者数据语义的不同, 对不同的变量可以适当加权
- 很难定义什么是“足够相似”, 答案的主观性很强

Requirements of Clustering in Data Mining

- 可伸缩性
- 能够处理各种不同类型的属性
- 能够发现任意形状的聚类
- 在决定输入参数的时候，对领域知识的需求要小
- 能够处理噪声和异常点
- 对输入数据的顺序部敏感
- 可以处理高维数据
- 可以和用户制定的限定条件相结合
- 可解释性和使用性好

Data Structures

- **Data matrix** (数据矩阵, 或称为对象属性结构) : 用p个变量 (也称为属性) 来表现n个对象, 例如用年龄, 身高, 性别, 种族等属性来表现对象“人”。这种数据结构是关系表的形式, 或者看为n*p维 (n个对象*p个属性) 的矩阵。

$$\begin{bmatrix} x_{11} & \dots & x_{1f} & \dots & x_{1p} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ x_{i1} & \dots & x_{if} & \dots & x_{ip} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ x_{n1} & \dots & x_{nf} & \dots & x_{np} \end{bmatrix}$$

Data Structures

- **Distance matrix** (距离矩阵, 或称为对象-对象结构) : 存储n个对象两两之间的距离, 表现形式是一个n*n维的矩阵。
- $d(i,j)$ 是对象i和对象j之间距离的量化表示, 通常它是一个非负的数值, 当对象i和j越相似, 其值越接近0; 两个对象越不同, 其值越大。既然 $d(i,j) = d(j,i)$, 而且 $d(i,i)=0$, 可以得到如下矩阵:

$$\begin{bmatrix} 0 & & & & \\ d(2,1) & 0 & & & \\ d(3,1) & d(3,2) & 0 & & \\ \vdots & \vdots & \vdots & & \\ d(n,1) & d(n,2) & \dots & \dots & 0 \end{bmatrix}$$

二模矩阵和单模矩阵

- 数据矩阵经常被称为二模 (two-mode) 矩阵，而距离矩阵被称为单模 (one-mode) 矩阵。这是因为前者的行和列代表不同的实体，而后的行和列代表相同的实体。
- 许多聚类算法以距离矩阵为基础。如果数据是用数据矩阵的形式表现的，在使用该类算法之前要将其转化为距离矩阵。
- 那么如何来估算不同类型数据的距离？

聚类分析中的数据类型

- Interval-scaled variables (区间标度)
- Binary variables (二元变量)
- Nominal, ordinal, and ratio variables (标称型、序数型和比例标度型)
- Variables of mixed types (混合型)

Interval-valued variables

- 区间标度变量的典型的例子包括重量和高度，经度和纬度坐标，以及大气温度等。
- 选用的度量单位将直接影响聚类分析的结果。例如，将高度的度量单位由“米”改为“英尺”，或者将重量的单位由“千克”改为“磅”，可能产生非常不同的聚类结构。
- 为了避免对度量单位选择的依赖，数据应当标准化。为了实现度量值的标准化，一种方法是将原来的度量值转换为无单位的值。

数据的标准化

- 给定一个变量 f 的度量值，可以进行如下的变换：
 - 计算平均的绝对偏差 (mean absolute deviation) S_f

$$s_f = \frac{1}{n}(|x_{1f} - m_f| + |x_{2f} - m_f| + \dots + |x_{nf} - m_f|)$$

其中 $m_f = \frac{1}{n}(x_{1f} + x_{2f} + \dots + x_{nf})$.

- 计算标准化的度量值，或z-score:

$$z_{if} = \frac{x_{if} - m_f}{s_f}$$

计算对象间的距离

- 通常使用明考斯基距离

$$d(i, j) = \sqrt[q]{(|x_{i1} - x_{j1}|^q + |x_{i2} - x_{j2}|^q + \dots + |x_{ip} - x_{jp}|^q)}$$

其中 $i = (x_{i1}, x_{i2}, \dots, x_{ip})$, $j = (x_{j1}, x_{j2}, \dots, x_{jp})$
分别代表两个 p -维的对象, q 是一个正整数

- $q = 1$ 的时候, d 称为曼哈顿距离

$$d(i, j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \dots + |x_{ip} - x_{jp}|$$

计算对象间的距离(Cont.)

■ 当 $q=2$ 表示欧几里得距离

$$d(i, j) = \sqrt{(|x_{i_1} - x_{j_1}|^2 + |x_{i_2} - x_{j_2}|^2 + \dots + |x_{i_p} - x_{j_p}|^2)}$$

■ 距离函数具有如下属性:

- n 1. $d(i, j) \geq 0$: 距离是一个非负的数值。
- n 2. $d(i, i) = 0$: 一个对象与自身的距离是0。
- n 3. $d(i, j) = d(j, i)$: 距离函数具有对称性。
- n 4. $d(i, j) \leq d(i, h) + d(h, j)$: 从对象I到对象j的直接距离不会大于途径任何其他对象的距离。

1. 可以对每个变量根据其重要性赋予一个权重

二元变量

- 二元变量只有两个状态：0或1
 - 0表示该变量为空
 - 1表示该变量存在
 - 例如，给出一个描述病人的变量smoker，1表示病人抽烟，而0表示病人不抽烟。
 - 像处理区间标度变量一样来对待二元变量会误导聚类结果，所以要采用特定的方法来计算其相异度。

二元变量可能性表

- 假设所有的二元变量有相同的权重，得到一个两行两列的可能性表
 - a 是对对象 i 和 j 值都为1的变量的数目
 - b 是在对象 i 中值为1，在对象 j 中值为0的变量的数目
 - c 是在对象 i 中值为0，在对象 j 中值为1的变量的数目
 - d 是在对象 i 和 j 中值都为0的变量的数目
 - 变量的总数是 $a+b+c+d$

		Object j		
		1	0	sum
Object i	1	a	b	$a+b$
	0	c	d	$c+d$
	sum	$a+c$	$b+d$	p

对称的二元变量

- 如果二元变量的两个状态有相同的权重, 那么该二元变量是对称的, 也就是两个取值0或1没有优先权。例如, “性别”
- 基于对称二元变量的相似度称为恒定的相似度, 即当一些或者全部二元变量编码改变时, 计算结果不会发生变化。
- 对恒定的相似度来说, 评价两个对象*i*和*j*之间距离的最著名的系数是简单匹配系数, 其定义如下:

$$d(i, j) = \frac{b + c}{a + b + c + d}$$

不对称的二元变量

- 如果两个状态的输出不是同样重要，那么该二元变量是不对称的。例如一个疾病检查的肯定和否定的结果。根据惯例，我们将比较重要的输出结果，通常也是出现几率较小的结果编码为1，而将另一种结果编码为0。
- 给定两个不对称的二元变量，两个都取值1的情况（正匹配）被认为比两个都取值0的情况（负匹配）更有意义。基于这样变量的相似度被称为非恒定的相似度。
- 对非恒定的相似度，最著名的评价系数是Jaccard系数，在它的计算中，负匹配的数目被认为是不重要的，因此被忽略。

$$d(i, j) = \frac{b + c}{a + b + c}$$

例子

■ Example

Name	Gender	Fever	Cough	Test-1	Test-2	Test-3	Test-4
Jack	M	Y	N	P	N	N	N
Mary	F	Y	N	P	N	P	N
Jim	M	Y	P	N	N	N	N

- gender is a symmetric attribute, 假设只计算非对称二元变量
- the remaining attributes are asymmetric binary
- let the values Y and P be set to 1, and the value N be set to 0

$$d(\text{jack}, \text{mary}) = \frac{0 + 1}{2 + 0 + 1} = 0.33$$

$$d(\text{jack}, \text{jim}) = \frac{1 + 1}{1 + 1 + 1} = 0.67$$

$$d(\text{jim}, \text{mary}) = \frac{1 + 2}{1 + 1 + 2} = 0.75$$

Nominal Variables (标称变量)

- 标称变量是二元变量的推广，它可以具有多于两个的状态值。例如，map_color是一个标称变量，它可能有五个值：红色，黄色，绿色，粉红色，和蓝色。
- 假设一个标称变量的状态数目是M。这些状态可以用字母，符号，或者一组整数（如1，2，...，M）来表示。
- 两个对象i和j之间的距离可以用简单匹配方法来计算：

$$d(i, j) = \frac{p - m}{p}$$

其中m是匹配的数目，即i和j取值相同的变量的数目；而 p是全部变量的数目。

Nominal Variables (标称变量)

- 另外一种方法是为每个状态创建一个二元变量，可以用二元变量来表示标称变量。
- 对一个有特定状态值的对象，对应该状态值的二元变量值置为1，而其余的二元变量值置为0。
- 例如，为了对标称变量map_color进行编码，对应于上面所列的五种颜色分别创建一个二元变量。如果一个对象是黄色，那么yellow变量被赋值为1，而其余的四个变量被赋值为0。
- 对于这种形式的编码，可以采用前面的方法来计算相异度。

Ordinal Variables (序数型变量)

- 序数型变量类似于标称变量，但序数型变量的M个状态是以有意义的序列排序的。例如，职称。

- 采用和区间标度变量类似的方法来计算对象的相异度：假设f是用于描述n个对象的一组序数型变量之一，关于f的相异度计算包括如下步骤：

- 1. 第i个对象的f值为 x_{if} ，变量f有 M_f 个有序的状态，对应于序列1, ..., M_f 。用对应的 r_{if} 代替 x_{if} ， $r_{if} \in \{1, \dots, M_f\}$ 。

- 2. 将每个变量的值域映射到 [0.0, 1.0] 上，通过用 z_{if} 代替 r_{if} 来实现。

$$z_{if} = \frac{r_{if} - 1}{M_f - 1}$$

- 3. 相异度的计算可以采用区间标度变量的方法，采用 z_{if} 作为第i个对象的f值。

Ratio-Scaled Variables (比例标度变量)

- 比例标度变量的典型例子包括细菌数目的增长，或者放射性元素的衰变
- 通常应用对数转换的方法：

$$y_{if} = \log(x_{if})$$

主要的聚类算法

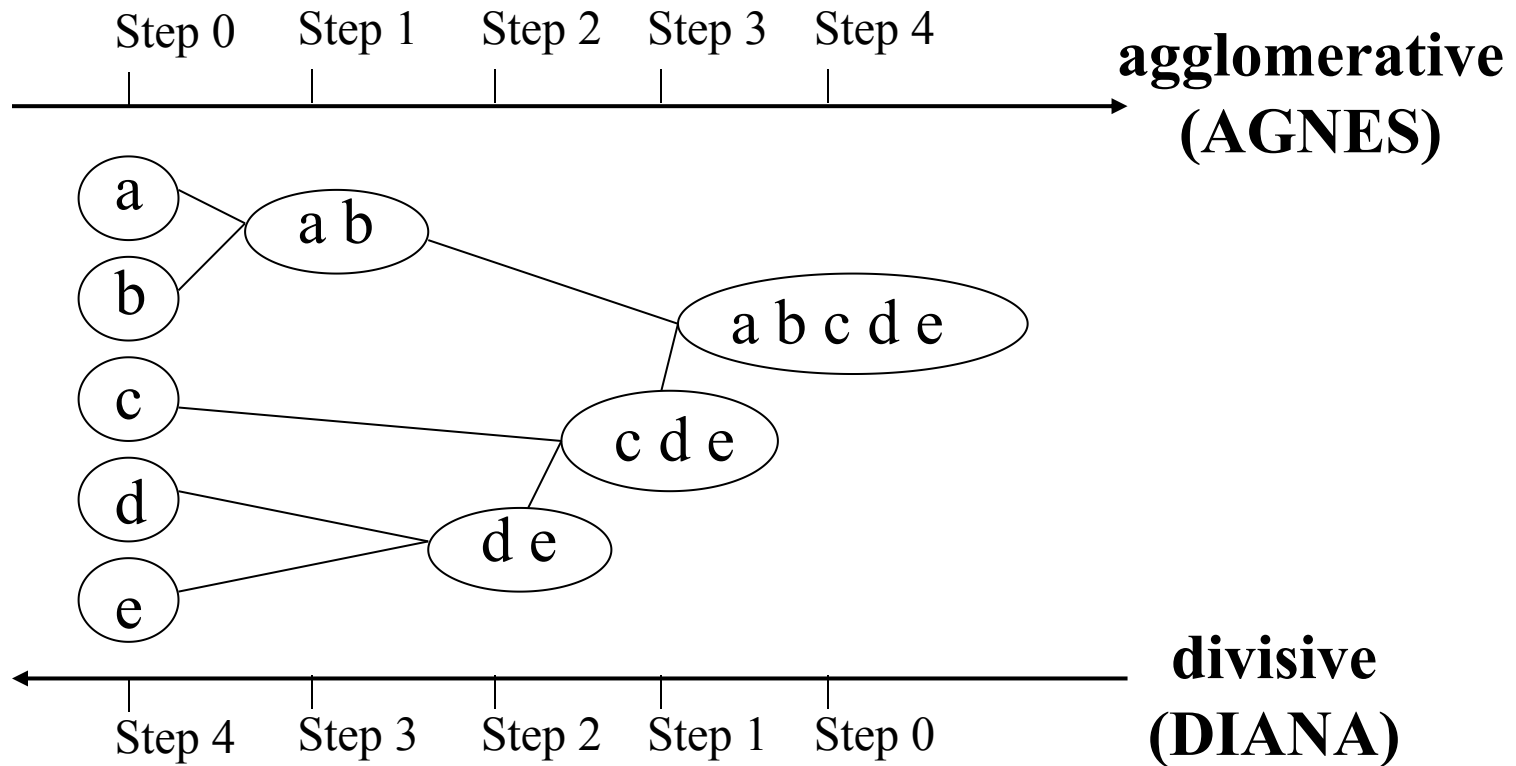
1、基于划分的方法

给定一个 n 个对象或元组的数据库，划分方法构建数据的 k 个划分，每个划分表示一个聚类，并且 $k \leq n$ 。也就是说，它将数据划分为 k 个组，同时满足如下的要求：（1）每个组至少包含一个对象；（2）每个对象必须属于且只属于一个组。

2、基于层次的聚类方法

主要思想是把数据对象排列成一个聚类树，在需要的层次上对其进行切割，相关联的部分构成一个cluster。基于层次的聚类方法有两种类型：

- (1) 聚合层次聚类。最初每个对象是一个cluster，然后根据它们之间的相似性，对这些原子的cluster进行合并。大多数层次方法属于这一类，它们的主要区别是cluster之间的相似性的定义不同。
- (2) 划分层次聚类，它与上面的过程正好相反。



-
- 用户可以指定算法终止的条件，例如，聚类的个数或每个**cluster**的半径低于某个阈值。
 - 弱点在于合并或分裂点的选取问题，因为一组对象一旦合并或分裂，就不能有**undo**的操作
 - 时间复杂度为 $O(N^2)$ ，对于处理大数据量有性能问题。

3、基于密度的方法

绝大多数划分方法基于对象之间的距离进行聚类。这样的方法只能发现凸状的簇，而在发现任意形状的簇上遇到了困难。

基于密度的聚类方法的主要思想是：只要临近区域的密度（对象或数据点的数目）超过某个阈值，就继续聚类。也就是说，对给定类中的每个数据点，在一个给定范围的区域中必须包含至少某个数目的点。这样的方法可以用来过滤“噪音”数据，发现任意形状的簇。

4、基于方格的方法

- 把多维数据空间划分成一定数目的单元，然后在这种数据结构上进行聚类操作。
- 该类方法的特点是它的处理速度，因为其速度与数据对象的个数无关，而只依赖于数据空间中每个维上单元的个数。

5、基于模型的方法

- (1) 神经网络方法
- (2) 统计的方法

基于划分的方法

- Partitioning method: Construct a partition of a database ***D*** of ***n*** objects into a set of ***k*** clusters, s.t., min sum of squared distance

$$\sum_{m=1}^k \sum_{t_{mi} \in K_m} (C_m - t_{mi})^2$$

- Given a *k*, find a partition of *k clusters* that optimizes the chosen partitioning criterion
 - Global optimal: exhaustively enumerate all partitions
 - Heuristic methods: *k-means* and *k-medoids* algorithms
 - *k-means* : Each cluster is represented by the center of the cluster
 - *k-medoids* : Each cluster is represented by one of the objects in the cluster

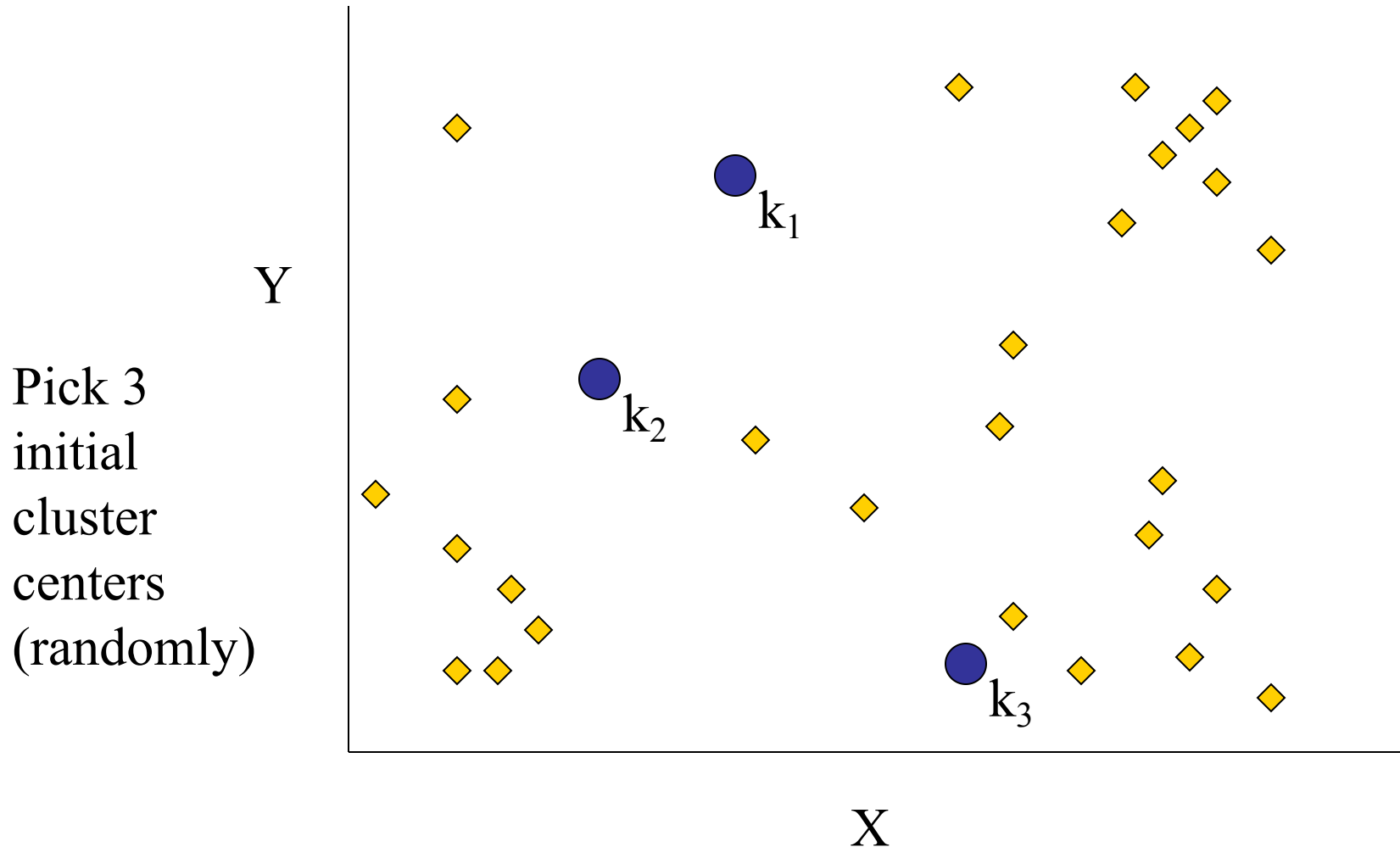
K-means算法

K-Means方法是MacQueen1967年提出的。给定一个数据集 X 和一个整数 K ($\leq n$)，K-Means方法是将 X 分成 K 个聚类并使得在每个聚类中所有值与该聚类中心距离的总和最小。

K-Means聚类方法分为以下几步:

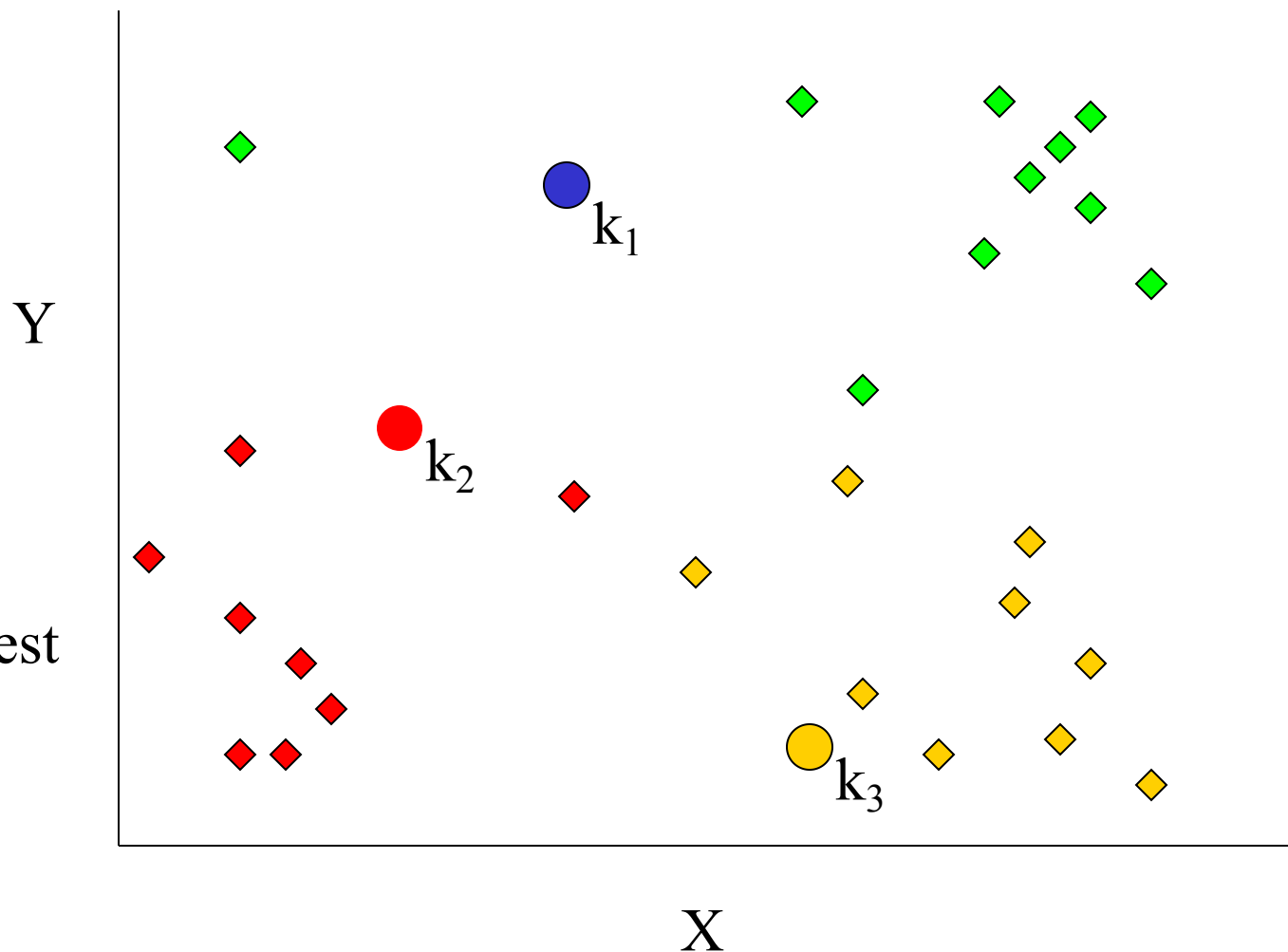
- [1] 给K个cluster选择最初的中心点，称为K个Means。
- [2] 计算每个对象和每个中心点之间的距离。
- [3] 把每个对象分配给距它最近的中心点做属的cluster。
- [4] 重新计算每个cluster的中心点。
- [5] 重复2，3，4步，直到算法收敛。

k -Means Example (I)



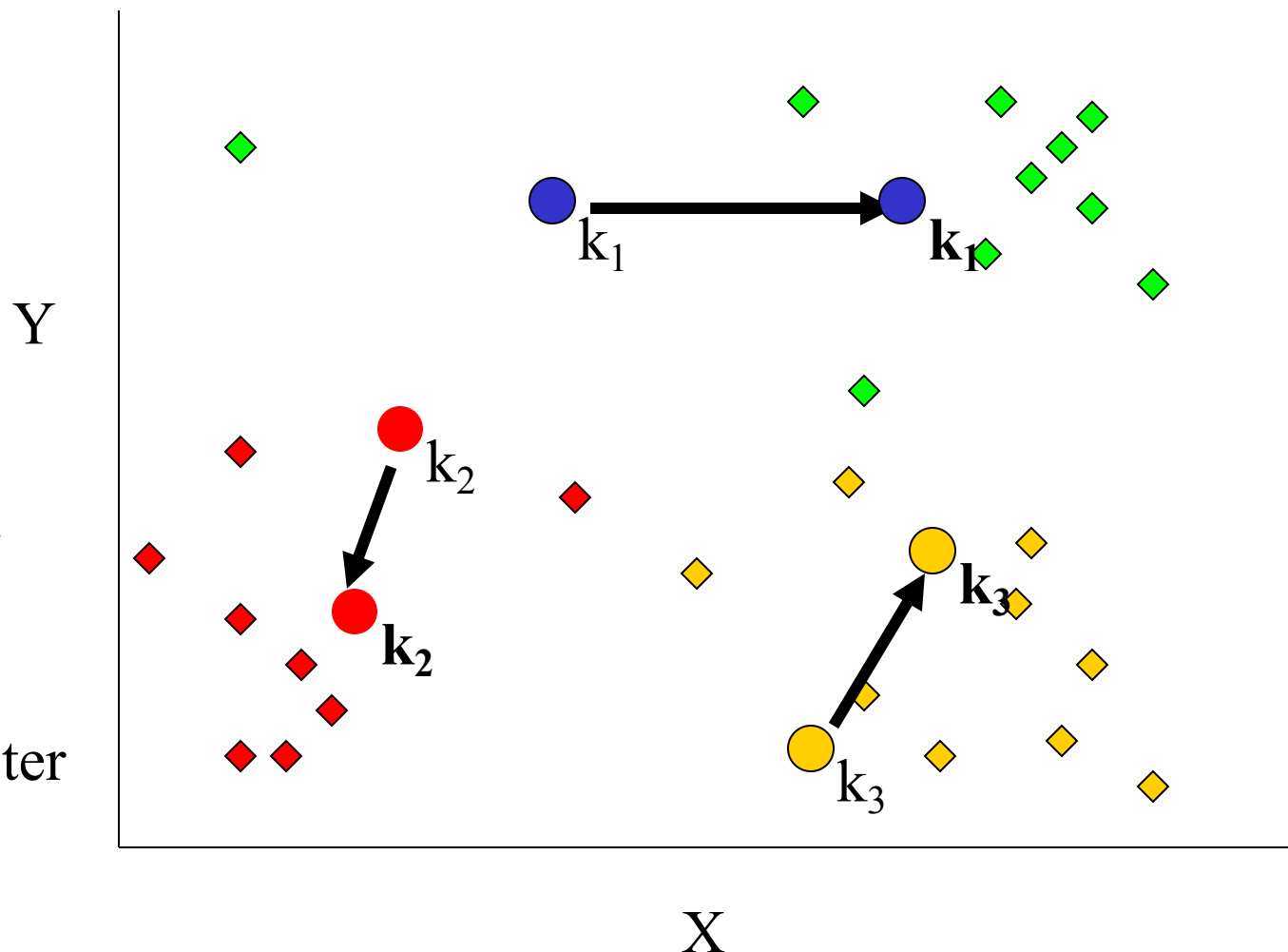
k -Means Example (II)

Assign
each point
to the closest
cluster
center



k -Means Example (III)

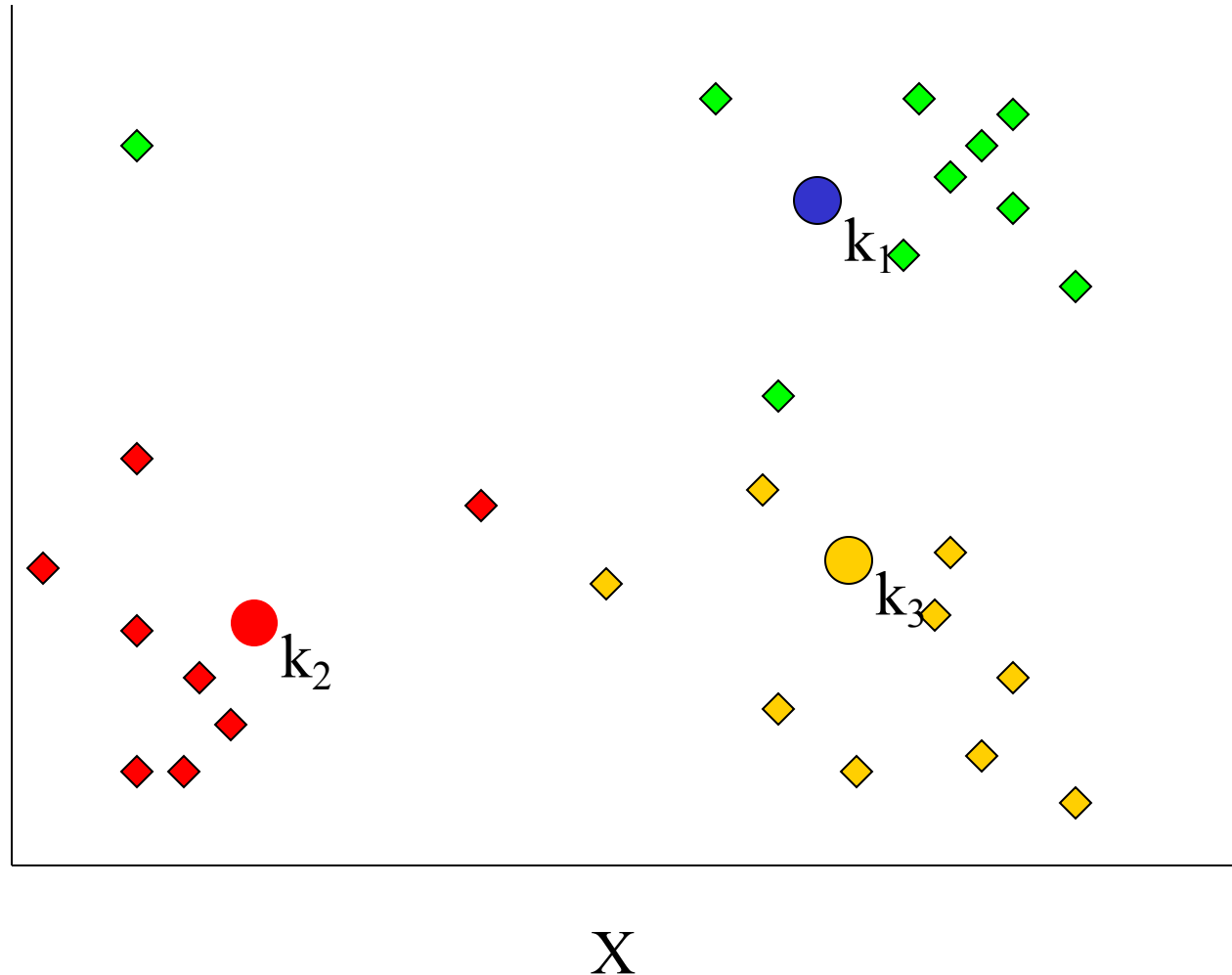
Move
each cluster
center
to the mean
of each cluster



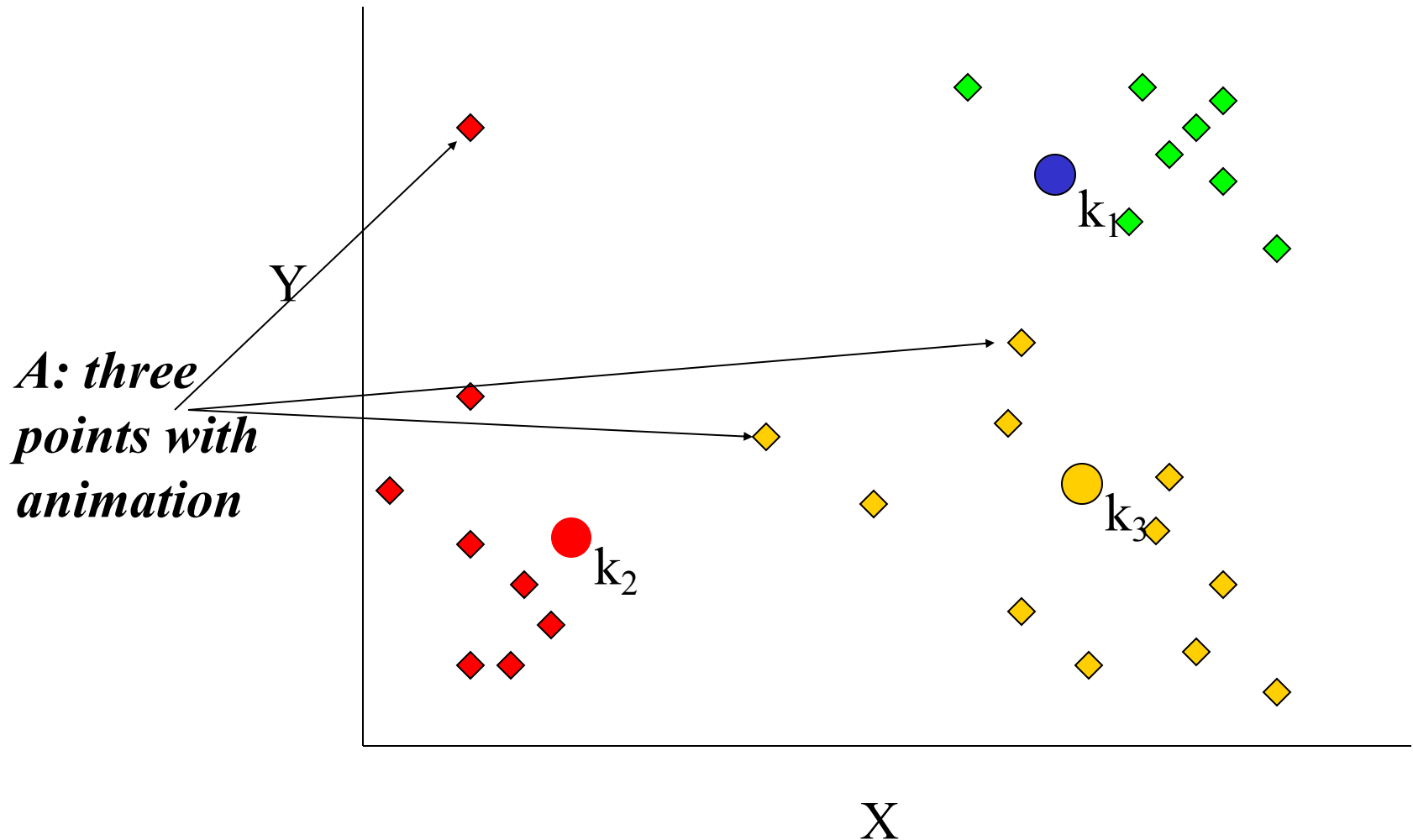
k -Means Example (IV)

Reassign
points
closest to a
different new
cluster center

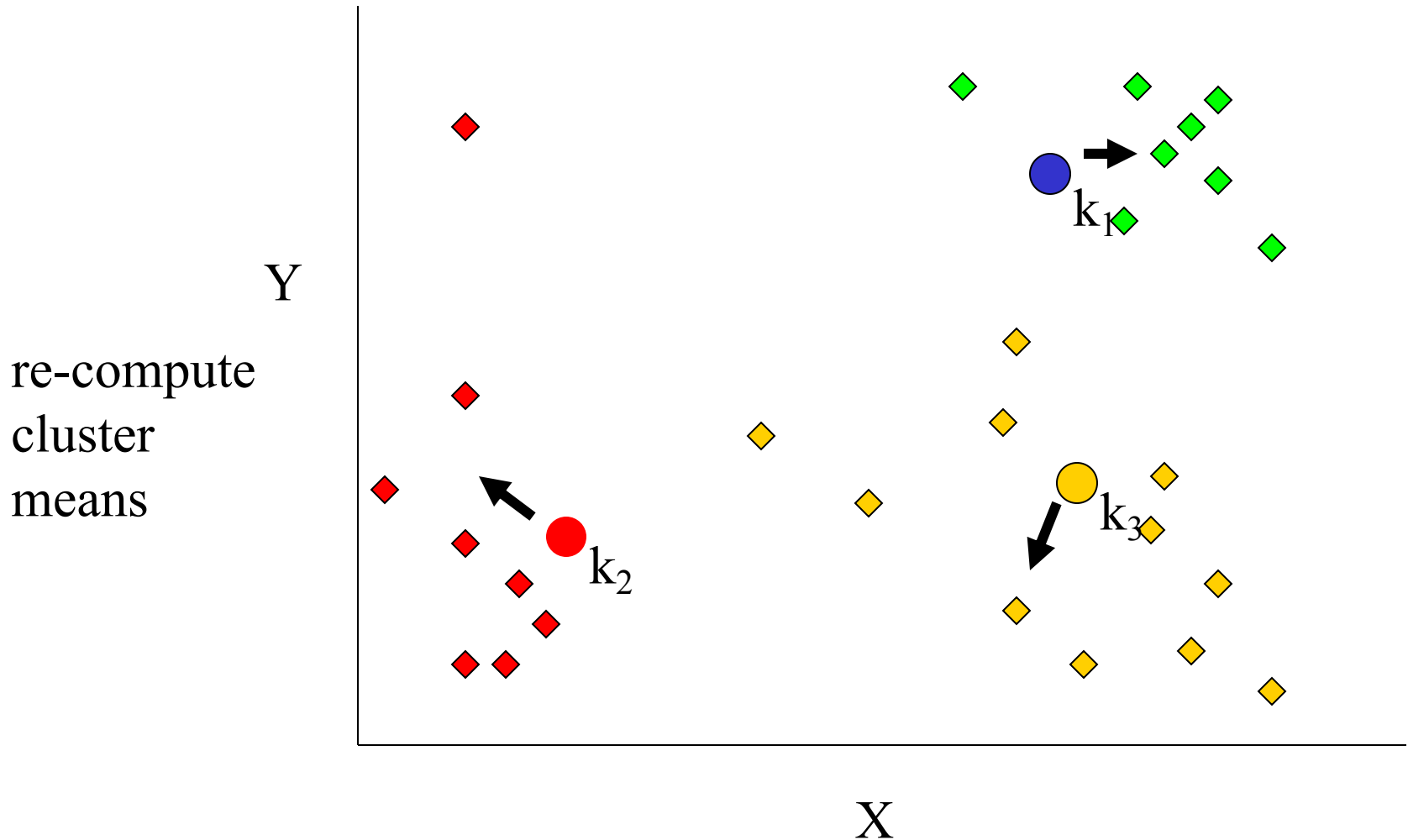
*Q: Which
points are
reassigned?*



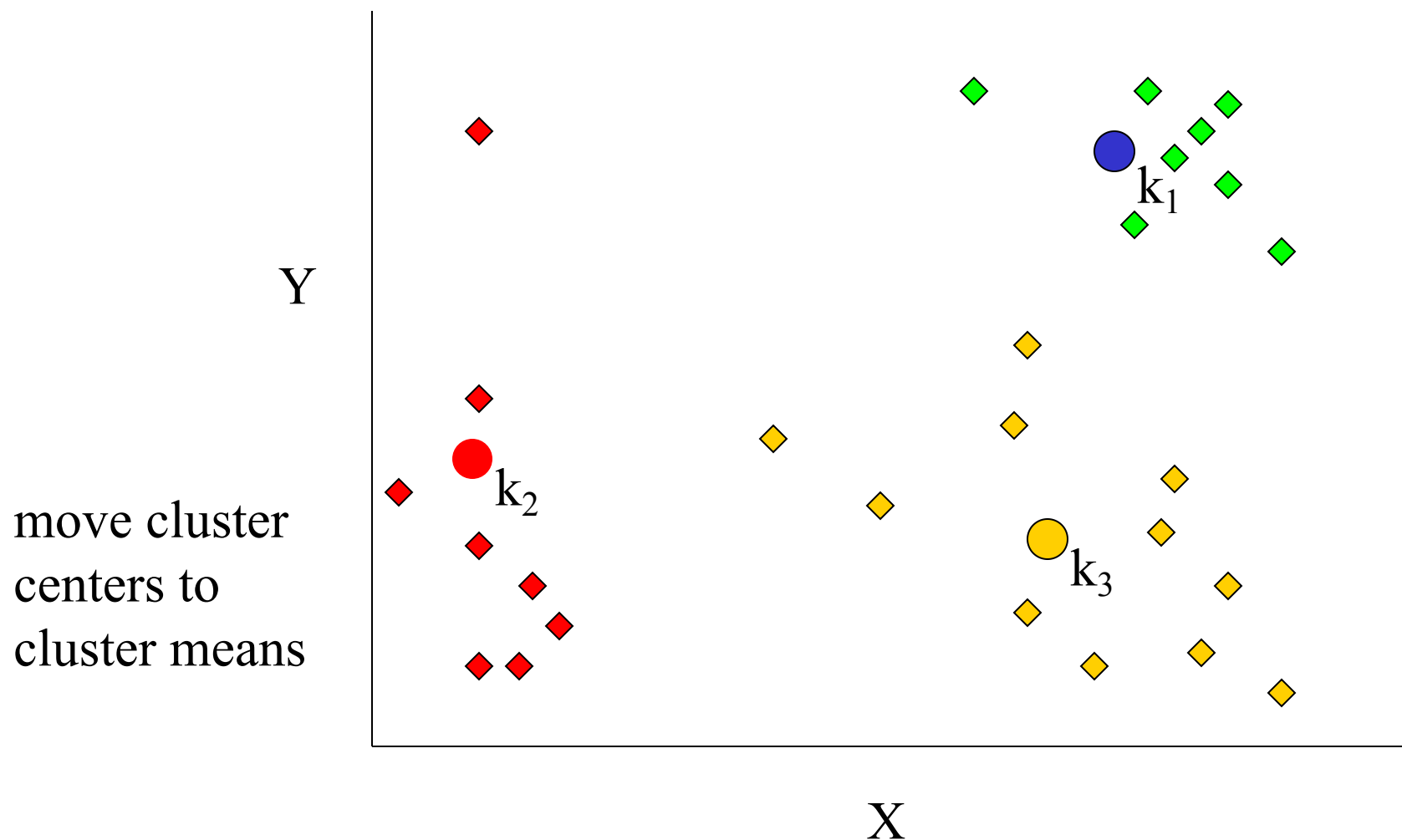
k -Means Example (V)



k -Means Example (VI)



k -Means Example (VII)



The K-Means Algorithm - Example [1]

- 把14个人分成3组
- 只有一个属性，年龄
- 初始的centroids 是 1, 20, , 40
- 右边的表是完成步骤1, 2后的结果

Cluster		C1	C2	C3
Centroid Value		1	20	40
P1	1	0	19	39
P2	3	2	17	37
P3	5	4	15	35
P4	8	7	12	32
P5	9	8	11	31
P6	11	10	9	29
P7	12	11	8	28
P8	13	12	7	27
P9	37	36	17	3
P10	43	42	23	3
P11	45	44	25	5
P12	49	48	29	9
P13	51	50	31	11
P14	65	64	45	

The K-Means Algorithm - Example [2]

- 重新计算 centroid , 得到5, 12, 和 48
- 重新计算每个实例与 3个Cluster的距离
- P5 更接近 C2
- 需要重新计算C1 和 C2的centroid, C3 没有变化不需要重新计算

Cluster		C1	C2	C3
Centroid Value		5	12	48
P1	1	4	11	47
P2	3	2	9	45
P3	5	0	7	43
P4	8	3	4	40
P5	9	4	3	39
P6	11	6	1	37
P7	12	7	0	36
P8	13	8	1	35
P9	37	32	25	11
P10	43	38	31	5
P11	45	40	33	3
P12	49	44	37	1
P13	51	46	39	3
P14	65	60	53	17

The K-Means Algorithm - Example [3]

- 3个Cluster的centroid是4, 11, 和 48
- 计算每个实例到Cluster的距离
- P4 更接近 C2
- 需要重新计算C1 和 C2的centroid, C3 没有变化不需要重新计算

Cluster		C1	C2	C3
Centroid Value		4	11	48
P1	1	3	10	47
P2	3	1	8	45
P3	5	1	6	43
P4	8	4	3	40
P5	9	5	2	39
P6	11	7	0	37
P7	12	8	1	36
P8	13	9	2	35
P9	37	33	26	11
P10	43	39	32	5
P11	45	41	34	3
P12	49	45	38	1
P13	51	47	40	3
P14	65	61	54	17

The K-Means Algorithm - Example [4]

- 3个Cluster的 centroid 是3, 10和 48
- 计算每个实例到 Cluster的距离
- 没有任何变化
- 算法不再迭代

Cluster		C1	C2	C3
Centroid Value		3	10	48
P1	1	2	9	47
P2	3	0	7	45
P3	5	2	5	43
P4	8	5	2	40
P5	9	6	1	39
P6	11	8	1	37
P7	12	9	2	36
P8	13	10	3	35
P9	37	34	27	11
P10	43	40	33	5
P11	45	42	35	3
P12	49	46	39	1
P13	51	48	41	3
P14	65	62	55	17

练习：假设数据挖掘的任务是将如下的6个点（用(x,y)代表位置）聚类为3个类。

A1(2,4),A2(3,7),B1(5,8),B2(9,5),C1(6,2),C2(4,9)，距离函数是曼哈顿函数。假设初始我们选择A1,B1,和C1为每个聚类的中心，用k-means算法来给出迭代2次后的三个聚类中心及其相应的聚类结果（中心的坐标值如是小数可向下取整）

■ 第一次迭代

- 选择初始的三个cluster的中心

Cluster1: A1(2,4)

Cluster2: B1(5,8)

Cluster3: C1(6,2)

- 计算3*6个距离

	A1(2,4)	A2(3,7)	B1(5,8)	B2(9,5)	C1(6,2)	C2(4,9)
A1(2,4)						
B1(5,8)						
C1(6,2)						

- 将每一个对象分配给距离它最近的中心

cluster1:

cluster2:

cluster3:

总结

■ 第一次迭代

- 选择初始的k个cluster的中心

Cluster1(C1):

Cluster2(C2):

...

Clusterk(C3):

- 计算k*n个距离

	X1	X2	...	Xn
C1				
C2				
...				
Ck				

- 将每一个对象分配给距离它最近的中心

C1:

C2:

...

Ck:

讨论

- 如何选择种子 C^0
- 可能的方法包括:
 - 随机的选择 C^0
 - 使用某种聚类算法的结果作为 C^0
 - **K-means** 采用的是**hill-climbing**, 只找到局部最优, 结果依赖于 C^0

- K-Means方法具有下面的优点

- (1) 对于处理大数据量具有可扩充性和高效率。算法的复杂度是 $O(tkn)$ ，其中 n 是对象的个数， k 是cluster的个数， t 是循环的次数，通常 $k, t \ll n$ 。
- (2) 可以实现局部最优化,如果要找全局最优，可以用退火算法或者遗传算法

- K-Means方法也有以下缺点

- (1) Cluster的个数必须事先确定，在有些应用中，事先并不知道cluster的个数。
- (2) K个中心点必须事先预定，而对于有些字符属性，很难确定中心点。
- (3) 不能处理噪音数据。
- (4) 不能处理有些分布的数据（例如凹形）

- K-Means方法的变种

- (1) K-Modes : 处理分类属性

- (2) K-Prototypes: 处理分类和数值属性

- (3) K-Medoids

它们与K-Means方法的主要区别在于:

- (1) 最初的K个中心点的选择不同。

- (2) 距离的计算方式不同。

- (3) 计算cluster的中心点的策略不同。

K-Medoids

- – instead of mean, use medians of each cluster
 - Mean of 1, 3, 5, 7, 9 is 5
 - Mean of 1, 3, 5, 7, 1009 is 205
 - Median of 1, 3, 5, 7, 1009 is 5
 - Median advantage: not affected by extreme values
- For large databases, use sampling

PAM: A K-medoids Method

- PAM: partitioning around Medoids
- 任意选择 k 个物体作为初始的 medoids
- Until no change, do
 - (Re)assign each object to the cluster to which the nearest medoid
 - Randomly select a non-medoid object o' , compute the total cost, S , of swapping medoid o with o'
 - If $S < 0$ then swap o with o' to form the new set of k medoids

Swapping Cost

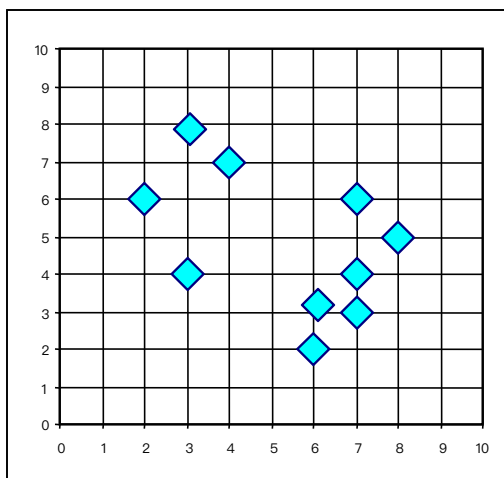
- 测试 o' 是否比 o 适合作为 medoid
- 采用下面的公式作为标准

$$E = \sum_{i=1}^k \sum_{p \in C_i} d(p, o_i)^2$$

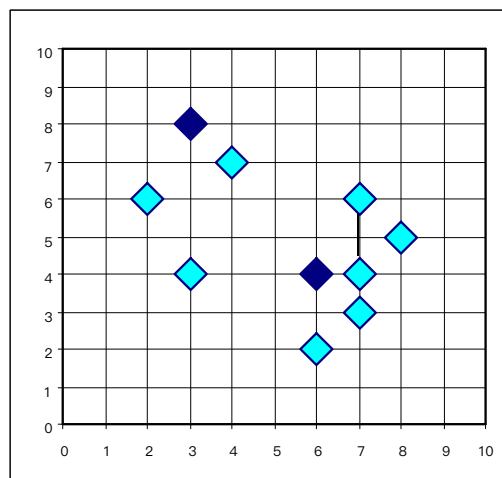
- 计算 $E_{o'} - E_o$
- 值为负数, 则可以交换

PAM: Example

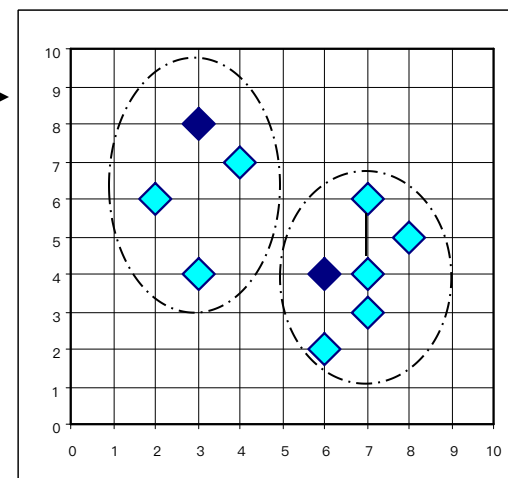
Total Cost = 20



Arbitrary
choose k
object as
initial
medoids



Assign
each remainin
g object to
nearest
medoids

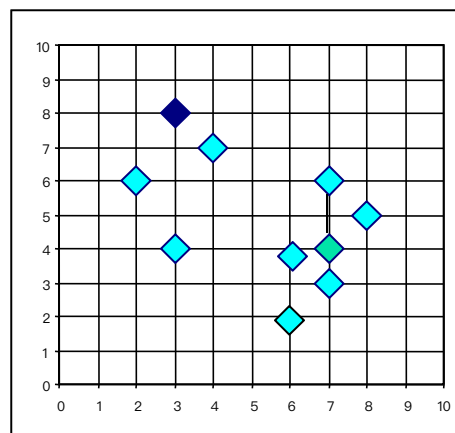


$K=2$

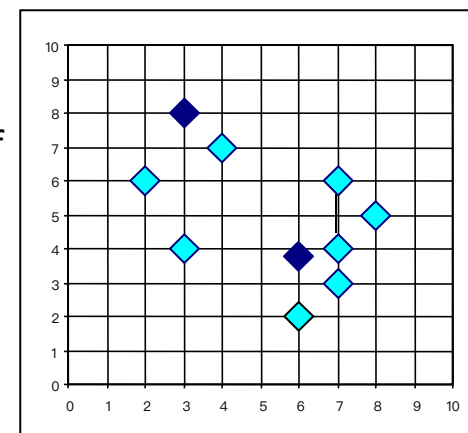
Do loop
Until no change

Swapping O
and O_{random}
If quality is
improved.

Total Cost = 26



Compute
total cost of
swapping



PAM的优缺点

- 有噪音和奇异点时，PAM比 k-means 鲁棒
- PAM 适用于小数据量，但不适合大数据量
 - $O(k(n-k)^2)$ for each iteration
- 基于采样: CLARA